

Remote Job Auto-Apply Pipeline

A structured-output LLM workflow that filters, scores, and applies — automatically

MALHAR SATISHKUMAR

JOB APPLICATION PROCESSING PIPELINE





The Problem: High-Effort, Mostly Wasted



Hours Lost

Applying to remote jobs is high-effort and mostly wasted — hours tailoring a resume for a listing that's already closed, or a role that's a bad fit.



Unstructured AI Output

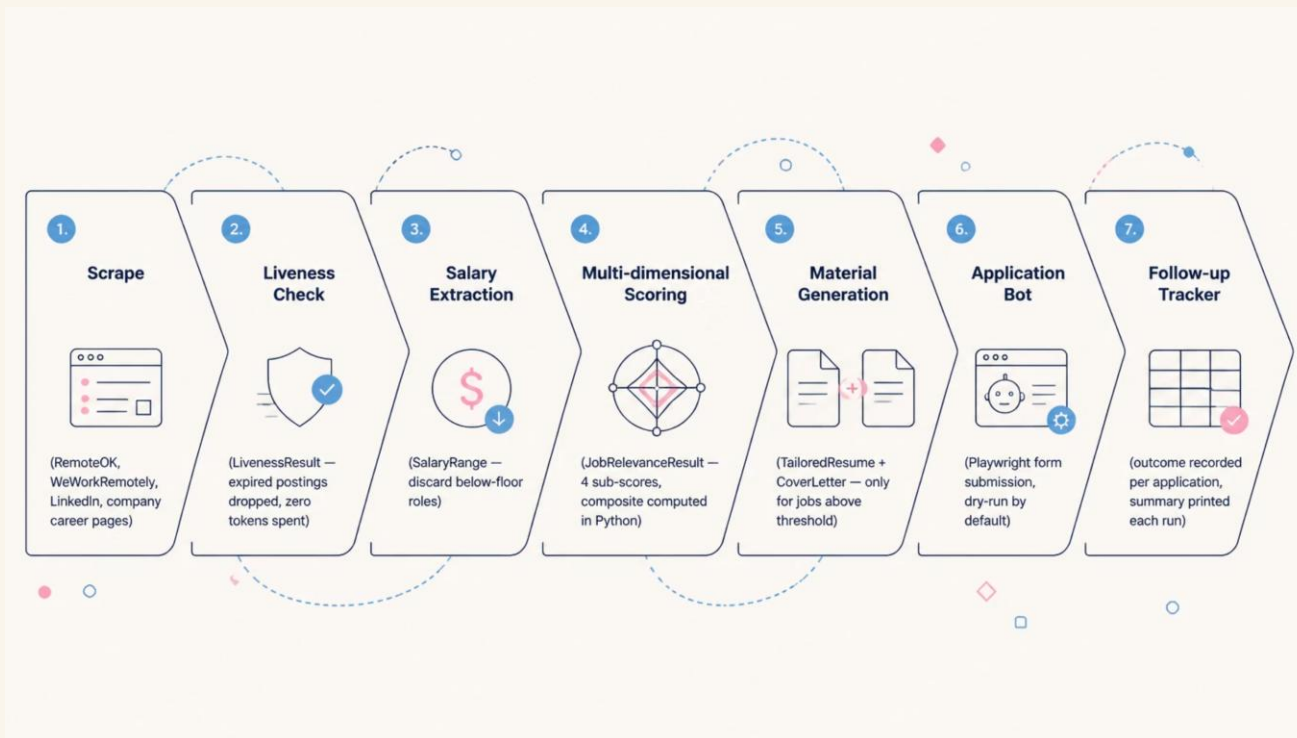
Automating it with a plain LLM gives you unstructured text you can't safely use in a pipeline.



This Project Solves Both

Dead listings are filtered before any AI tokens are spent, and every AI output is a validated, typed Pydantic model.

How the Pipeline Works



Each step in the pipeline is a gated stage — jobs that fail a gate are dropped immediately, ensuring no wasted computation or token expenditure downstream.

Every AI Step Returns a Validated Model

The LLM never returns free text — it returns a typed Pydantic object. If the output doesn't fit the schema, the pipeline fails loudly instead of passing bad data downstream.

Schema Reference

Schema	Key Fields
LivenessResult	status (active / expired / uncertain), reason
SalaryRange	is_disclosed, min_salary, max_salary, currency, period
JobRelevanceResult	role_match, level_fit, growth_potential, remote_alignment (0–1 each) + computed score
CoverLetter	opening, body, closing
TailoredResume	summary, skills[], experience[], education[]

Scoring Formula

The LLM scores the dimensions. The weights and arithmetic live in your code, not the prompt.

```
score = round(  
    (role_match × 0.45  
    + level_fit × 0.30  
    + growth_potential × 0.15  
    + remote_alignment × 0.10) × 10  
)
```

Smart Gates Save Tokens (and Time)

Three-gate design — each gate only runs if the previous one passes:

Gate	Schema	What it prevents
Liveness	<code>LivenessResult</code>	Scoring a closed posting
Salary floor	<code>SalaryRange</code>	Scoring an underpaying role
Relevance threshold	<code>JobRelevanceResult</code>	Generating materials for a poor fit

Other Design Decisions

Composite Score in Python

Not prompt-engineered. Weights are code, not magic words.

`dry_run=True` by Default

Playwright fills forms but never submits without explicit opt-in.

Outcome Tracking

Each application records outcome (`pending` → `interview` / `rejected` / `offer`), enabling future response-rate analytics.

Follow-up Automation

7-day follow-up date written at application time; surfaced automatically each run.

Live Terminal Dashboard

What the TUI shows (`python tui.py --demo`):

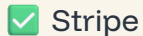
→ Full-Screen Pipeline Monitor

Staged reveals per job: Liveness → Salary → Scoring (live bar charts per sub-score) → Materials

→ Running Footer

Scanned / dropped at liveness / scored / materials generated
— every value rendered passes through the same Pydantic models the pipeline uses

Sample Run Over 3 Jobs



Stripe

Active, \$180k–\$240k/yr, score **8/10** (high fit) → resume + cover letter generated



Acme Corp

Expired at liveness → dropped, scorer never called



Vercel

Active, \$130k–\$170k/yr, score **7/10** (medium fit) → resume + cover letter generated



Tech stack: openai · pydantic · sqlalchemy · playwright · rich · requests | **Source:** github.com/jaimalharsk/dev-lab → [experiments/local-ai/remote-job-autoapply-mvp](#)